

Сейчас преподавательский хаб — **разработческий MVP**: он работает, но требует кодовую базу, Node.js и запуск из терминала. Для обычного преподавателя это неприемлемо — этот слой ещё нужно упаковать.

Может ли телефон преподавателя быть хабом?

Только телефон + PWA — нет

Преподавательский хаб должен:

- рекламировать BLE-сервис **AttendPro**;
- работать как BLE peripheral;
- предоставлять GATT-характеристики для чтения якоря, записи отметки и отправки квитанции.

Web Bluetooth умеет только роль central/GATT-клиента. Браузер не может рекламировать BLE-сервис и стать GATT-сервером.

Поэтому:

- студентский телефон может работать через PWA/Bluefy;
- преподавательский телефон не может быть хабом через обычную веб-страницу.

Телефон преподавателя с нативным приложением — может

Можно написать приложение с:

- **CBPeripheralManager** для iPhone;
- **BluetoothLeAdvertiser** и GATT Server для Android.

Но существующий Capacitor BLE-плагин поддерживает только central-роль. Понадобится собственный нативный плагин. В текущем MVP этого нет.

Телефон как пульт управления — может

Телефон преподавателя может открыть веб-консоль и управлять уже работающим хабом:

Телефон преподавателя → веб-консоль → Mac/ESP32-хаб

Но сам Bluetooth при этом предоставляет Mac или ESP32.

Как это должно выглядеть для преподавателя на компьютере

Есть два нормальных продуктовых варианта.

Вариант 1. Приложение AttendPro Hub для Mac/Windows

Это самый быстрый путь из текущего кода.

Преподаватель получает, например:

```
AttendPro Hub.dmg
```

Один раз устанавливает приложение и дальше:

1. Запускает **AttendPro Hub** из Launchpad.
2. macOS один раз просит разрешить Bluetooth.
3. Приложение само запускает встроенный Node-хаб в фоне.
4. Автоматически открывается окно:

```
AttendPro – Консоль преподавателя
```

5. Преподаватель выбирает занятие.
6. Нажимает «Начать пару».
7. Создаёт приглашения.
8. Видит список отметившихся.
9. Нажимает «Завершить пару».
10. Закрывает приложение — хаб останавливается.

Никаких терминалов, **npm**, исходников и **admin-token**.

Внутри **.app** будут упакованы:

```
AttendPro Hub.app
├── Node runtime
├── hub/server.cjs
├── bleno
├── student PWA
├── web-консоль
└── локальные данные хаба
```

Данные будут храниться не в репозитории, а, например:

```
~/Library/Application Support/AttendPro Hub/
```

Преподаватель этого не видит.

Что показывается в интерфейсе

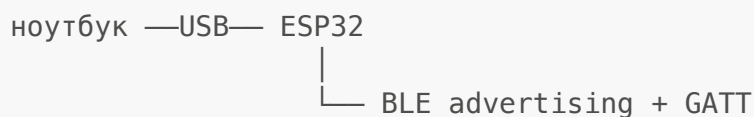
AttendPro Hub	BLE: готов
Пара: Алгоритмы	
Аудитория: 312	
[Начать занятие]	
Студенты:	
1. Иван Иванов	10:02 ✓
2. Мария Петрова	10:03 ✓
[Создать приглашение]	
[Завершить занятие]	

Формально внутри может открываться браузерная страница, но для пользователя это будет выглядеть как обычное приложение.

Вариант 2. ESP32-свисток + преподавательская PWA

Это лучший конечный вариант из концепции v4.

Преподаватель получает маленький USB-свисток ESP32:



Действия преподавателя:

1. Вставляет AttendPro-свисток в USB.
2. Открывает в Chrome страницу преподавателя.
3. Нажимает «Подключить хаб».
4. Chrome показывает системный выбор USB-устройства.
5. Преподаватель выбирает **AttendPro Hub**.
6. Нажимает «Начать пару».

В этом варианте:

- Node.js не нужен;
- терминал не нужен;
- кодовая база не нужна;
- Bluetooth компьютера не используется;
- преподавательская консоль может быть обычной PWA;
- GATT-server и реклама работают внутри ESP32;
- одинаково работает на Mac, Windows и Linux.

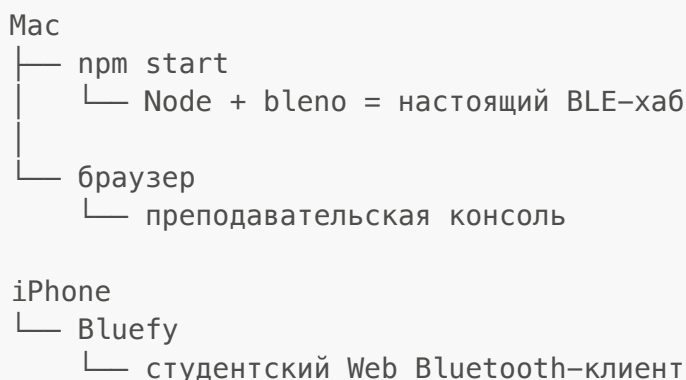
Браузер разговаривает со свистком через Web Serial:



Это как раз рекомендованный вариант Б в [attendpro-ble-first-v4.md](#).

Что мы тестируем сейчас

Сейчас архитектура выглядит так:



Веб-страница преподавателя **не является хабом**. Она лишь управляет Node-процессом, который работает рядом на Mac.

Текущий запуск через терминал нужен для проверки:

- BLE-рекламы;
- GATT-протокола;
- подписей;
- квитанций;
- поведения iPhone/Android;
- преподавательского интерфейса.

После проверки этот же Node-хаб можно упаковать в [.app](#), не меняя протокол.

Что рекомендую для MVP

Ближайший пилот

Сделать **AttendPro Hub.app** для Mac:

- самый быстрый путь;
- уже есть вся серверная логика;
- уже работает BLE через bleno;
- преподавателю останется только нажать на иконку.

Следующий этап

ESP32-свисток + чистая Teacher PWA:

- без установки приложения на компьютер преподавателя;
- без зависимости от Bluetooth-драйверов;
- кроссплатформенно;
- лучше масштабируется на аудитории.

Итог

Сценарий	Реализуемо?
Телефон преподавателя + чистая PWA как BLE-хаб	✗
Телефон преподавателя + нативное приложение	✓, но ещё не написано
Mac + упакованный AttendPro Hub.app	✓, быстрее всего
Компьютер + ESP32 + Teacher PWA	✓, лучший конечный вариант
Телефон преподавателя как пульт для Mac/ESP32-хаба	✓

То есть для законченного ближайшего MVP логичная схема: **студенты — PWA/Bluefy, преподаватель — AttendPro Hub.app на Mac**. А затем заменить нативную радиочасть преподавателя на ESP32, оставив ту же браузерную консоль.